

CVE-2024-3094

CVE-2024-3094: Supply Chain Compromise via Social Engineering and Build System Manipulation in XZ Utils

A Technical Analysis with Emphasis on Identity and Access Management Implications

Nikolaos Natsopoulos

IAM Consultant | Independent Researcher

DOI: 10.5281/zenodo.20700609
2026

CVE	CVSS Score	Affected Software	Discovery Date
CVE-2024-3094	10.0 (Critical)	XZ Utils 5.6.0, 5.6.1	29 March 2024

Keywords: supply chain attack, CVE-2024-3094, XZ Utils, liblzma, OpenSSH, backdoor, identity and access management, Linux security, social engineering, open-source integrity, IFUNC, build system injection

Abstract

In March 2024, a sophisticated supply chain attack targeting XZ Utils (CVE-2024-3094) was discovered moments before it could propagate to stable Linux distributions worldwide. The attack, assigned the maximum Common Vulnerability Scoring System (CVSS) score of 10.0, involved a fictitious contributor identity operating under the alias JiaT75 (Jia Tan) who spent approximately three years infiltrating the XZ Utils project through legitimate contributions, coordinated social engineering, and eventual acquisition of co-maintainer status. Once in a position of trust, the actor injected a malicious object file into the project's official release tarballs via a modified Autotools build script. The payload registered GNU C Library (glibc) Indirect Function (IFUNC) resolvers that silently overrode the **RSA_public_decrypt()** function at runtime, enabling unauthenticated remote code execution as root on any affected system with OpenSSH exposed to the network.

This report presents a comprehensive technical analysis of the attack organised around four research questions: what the threat actor did, how the attack was technically executed, why it was carried out, and what the consequences would have been had it succeeded. The report introduces a purpose-built analysis tool, the **CVE-2024-3094 Research Toolkit**, developed as part of this study to evaluate system exposure and visualise the dependency chain exploited by the attack. Three representative system profiles are analysed using the tool, demonstrating that vulnerability was both version-specific and architecture-specific.

A dedicated section analyses the attack from an Identity and Access Management (IAM) perspective, arguing that the attack represents a novel threat model not addressed by conventional IAM frameworks: the compromise of the authentication mechanism itself, rather than the credentials or authorisations it governs. The analysis demonstrates that controls including Multi-Factor Authentication, Privileged Access Management, Zero Trust Network Access, and Role-Based Access Control would have been entirely ineffective against this specific attack vector. The report concludes with recommendations for IAM practitioners and open-source security stakeholders.

Table of Contents

Abstract.....	2
Table of Contents	3
1. Introduction	6
1.1 Background and Motivation.....	6
1.2 Research Questions	7
1.3 Methodology	7
1.3.1 Literature Review.....	7
1.3.2 Tool-Based Configuration Analysis.....	7
1.3.3 Scope and Limitations	8
2. Background	9
2.1 XZ Utils and liblzma	9
2.2 OpenSSH and Its Relationship to liblzma.....	9
2.3 The Build System: Autotools and the Role of Configure Scripts	10
2.4 Threat Landscape Context.....	10
3. What Did the Threat Actor Do?.....	12
3.1 Identity: JiaT75 / Jia Tan	12
3.2 Phase 1: Account Creation and Initial Contributions (Jan 2021 – Nov 2022).....	12
3.3 Phase 2: Social Engineering Campaign (May – Dec 2022)	12
3.4 Phase 3: Position of Trust (Dec 2022 – Dec 2023).....	13
3.5 Phase 4: Payload Insertion (Jan – Mar 2024)	13
4. How Was It Done?.....	15
4.1 Payload Delivery: Build System Injection	15
4.2 The Three-Stage Injection Process	15
Stage 1: Configure Script Execution	15
Stage 2: Object File Extraction	16
Stage 3: IFUNC Resolver Registration	16
4.3 The Backdoor Hook: RSA_public_decrypt()	16
4.4 The Five Exploitation Conditions	17
4.5 Evasion Techniques.....	17
4.6 Tool-Based Configuration Analysis	18
5. Why Was It Done?.....	21
5.1 Nature of the Backdoor: Espionage, Not Crime.....	21
5.2 Technical Attribution Evidence	21
5.3 Strategic Target Assessment	21
5.4 Current Attribution Status	22

6. What Would Have Happened?	23
6.1 Scope of Exposure at Time of Discovery	23
6.2 Attack Capability If Deployed	24
6.3 Systemic Impact Scenarios	24
Scenario A: Targeted Espionage (Assessed Most Likely)	24
Scenario B: Infrastructure Disruption	24
Scenario C: Key Compromise	24
6.4 Cascading Supply Chain Effects	25
7. Identity and Access Management Implications	26
7.1 The Failure Mode This Attack Represents	26
7.2 The Attack Chain as a Layered IAM Problem	26
7.3 Software Supply Chain as an Identity Problem	27
7.4 Privileged Access Management Analysis	27
7.5 IAM Control Effectiveness Analysis	28
7.6 Implications for IAM Practitioners	29
Software Supply Chain as IAM Scope	29
Authentication Stack Integrity	29
Anomaly Detection as an IAM Control	29
Vendor and Dependency Risk Management	29
7.7 Regulatory and Compliance Dimensions	29
8. Community and Institutional Response	31
8.1 Immediate Technical Response (March – April 2024)	31
8.2 Medium-Term Reforms (2024–2025)	31
8.3 What Has Not Been Fixed	32
9. Conclusions	33
9.1 Summary of Findings	33
What the Threat Actor Did	33
How the Attack Was Technically Executed	33
Why the Attack Was Carried Out	33
What Would Have Happened	33
9.2 IAM-Specific Conclusions	33
9.3 Tool Contribution	34
9.4 Future Work	34
Appendix A: Affected Distribution and Version Matrix	35
Appendix B: MITRE ATT&CK Technique Mapping	36
Appendix C: CVE-2024-3094 Research Toolkit	37
C.1 Overview	37
C.2 System Vulnerability Checker	37
Input Commands	37

Evaluation Logic37

C.3 Attack Surface Visualiser38

C.4 Example System Profiles38

References39

Primary Sources39

Academic Sources39

Technical Analyses39

Attribution and Social Engineering Analyses40

Vendor Advisories and Distribution Notices40

Frameworks and Standards40

1. Introduction

1.1 Background and Motivation

Software supply chain attacks represent one of the most significant and underanalysed threat vectors in contemporary cybersecurity. Unlike direct attacks on target systems, supply chain attacks compromise components trusted by those systems - libraries, build tools, update mechanisms, and the maintainers who manage them. The transitive trust that makes software ecosystems efficient becomes, in this context, a structural vulnerability.

On 29 March 2024, Andres Freund, a Microsoft engineer working on PostgreSQL, published a disclosure to the oss-security mailing list documenting a backdoor embedded in XZ Utils versions 5.6.0 and 5.6.1. ^[1] XZ Utils is a data compression utility present on virtually every Linux system. The backdoor, assigned **CVE-2024-3094**, would have granted its operator unauthenticated remote code execution as root on any affected system running OpenSSH. It was discovered approximately five weeks before the backdoored versions would have reached stable, widely deployed Linux distributions.

The attack is notable for several reasons. First, it was not a technical exploit of a software vulnerability in the conventional sense, it was the culmination of a nearly three-year social engineering operation that resulted in the attacker gaining legitimate co-maintainer status on the project. Second, it targeted not just a library, but the build system used to produce that library's official releases, creating a divergence between the publicly reviewable source code and the distributed binaries. Third, the technical sophistication of the payload, involving IFUNC resolver manipulation and multi-stage obfuscation, indicates resources and capabilities associated with nation-state actors.

Existing published analyses of CVE-2024-3094 have focused primarily on the technical mechanics of the payload and the social engineering timeline. A significant gap exists in the literature: a systematic analysis of the attack from an Identity and Access Management (IAM) perspective. This gap is particularly meaningful because the attack's most profound implication for security practitioners is that it compromised the authentication mechanism itself - rendering the entire edifice of IAM controls built above that mechanism ineffective.

This report addresses that gap. It provides a complete technical analysis of the attack and situates that analysis within an IAM framework, identifying which controls would and would not have been effective, and what this implies for how IAM programmes must evolve to address supply chain threats.

1.2 Research Questions

This report is organised around four primary research questions:

1. **What** did the threat actor do? A chronological analysis of the actor's actions, from identity establishment through social engineering to payload insertion.
2. **How** was the attack technically executed? A technical decomposition of the build system injection, IFUNC hooking mechanism, and exploitation conditions.
3. **Why** was the attack carried out? An assessment of intent and attribution evidence.
4. **What would have happened** had it succeeded? An impact analysis covering scope of exposure, attack capability, and systemic consequences.

1.3 Methodology

1.3.1 Literature Review

This study is grounded in a systematic review of primary and secondary sources. Primary sources consulted include the original oss-security disclosure by Andres Freund, the NIST National Vulnerability Database entry for CVE-2024-3094, vendor security advisories from Red Hat and Debian, and the official statement from XZ maintainer Lasse Collin. Secondary sources include peer-reviewed publications on arXiv, technical analyses published by Kaspersky Securelist, JFrog Security Research, Datadog Security Labs, Elastic Security Labs, and ReversingLabs, as well as community-maintained technical documentation.

Framework references include NIST Special Publication 800-161r1 on Supply Chain Risk Management, the MITRE ATT&CK Enterprise framework, and the EU Cyber Resilience Act. Full references are provided in the References section.

1.3.2 Tool-Based Configuration Analysis

A purpose-built analysis tool, the **CVE-2024-3094 Research Toolkit**, was developed as an original contribution to this study. The tool accepts output from three standard Linux diagnostic commands and evaluates the five conditions required for exploitation of CVE-2024-3094, producing: (1) a binary vulnerability determination; (2) a per-condition breakdown with technical justification; and (3) a system-specific dependency chain visualisation reflecting the actual libraries present on the analysed system.

Three representative system profiles were analysed using the tool:

- **Profile A, Fully Vulnerable:** Debian Sid with XZ Utils 5.6.1, libsystemd-patched OpenSSH, glibc, and an active SSH daemon. All five exploitation conditions satisfied.
- **Profile B, Safe by Version:** Ubuntu LTS with XZ Utils 5.4.6. The XZ version condition fails at the first check; liblzma and libsystemd are additionally absent from the sshd dependency chain.
- **Profile C, Architecturally Protected:** Arch Linux with XZ Utils 5.6.0 (the backdoored version). The XZ version condition fails; however, the attack chain additionally breaks at libsystemd because Arch Linux does not apply the sd_notify patch to OpenSSH, meaning liblzma is never linked into sshd regardless of version.

Tool source code and usage documentation are provided in Appendix C. Binary and runtime analysis data referenced in Sections 4 and 5 derives from published primary sources, cited accordingly.

1.3.3 Scope and Limitations

Full runtime exploitation cannot be reproduced: the Ed448 cryptographic private key required to trigger the backdoor's RCE payload has not been publicly disclosed. Tool-based analysis evaluates static configuration conditions rather than runtime behaviour. Attribution analysis is conducted exclusively based on open-source intelligence; no classified or non-public sources were consulted. The tool produces analytical determinations based on textual pattern matching command output and should be used as a research instrument rather than a production security scanner.

2. Background

2.1 XZ Utils and liblzma

XZ Utils is a free, open-source file compression utility implementing the Lossless Data Compression Algorithm (LZMA) and its successor LZMA2. It is functionally equivalent to tools such as gzip and bzip2, producing compressed archives with the **.xz** extension. XZ Utils is present on virtually every Linux system: it is used by package managers to compress software packages, by the Linux kernel build process to compress initramfs images, by software distributors to produce release tarballs, and by system administrators for routine data management.

The core component of XZ Utils is **liblzma**, a shared library that exposes the compression and decompression functionality to other programs. liblzma is a dependency not only on applications that explicitly handle XZ-compressed data, but also on certain Linux distributions of **libsystemd**, the library component of systemd, the dominant Linux init system. This indirect dependency chain (liblzma ← libsystemd ← OpenSSH) is the critical path through which the backdoor was able to affect the SSH daemon.

The versions of XZ Utils containing the backdoor are **5.6.0** (released 24 February 2024) and **5.6.1** (released 9 March 2024). Prior versions, including the widely deployed 5.4.x series, are not affected. ^[2]

XZ Utils was maintained almost entirely by a single volunteer, Lasse Collin, a Finnish developer who had worked on the project since its inception over a decade before the attack. This 'bus factor of one', the condition in which an entire project depends on the continued availability of a single individual was not a technical vulnerability but a structural one, and it was this structural vulnerability that the threat actor targeted.

2.2 OpenSSH and Its Relationship to liblzma

OpenSSH is the implementation of the Secure Shell (SSH) protocol used on the majority of Linux servers worldwide. It provides encrypted remote access, enabling system administrators to manage servers securely over untrusted networks. The **sshd** daemon, the server component, listens on TCP port 22 and handles authentication, session management, and encrypted communication.

Under standard upstream OpenSSH, **sshd** has no dependency on liblzma. The dependency was introduced by certain Linux distributions, most notably Debian and its derivatives, through a non-upstream patch that adds support for **sd_notify**, a systemd mechanism that allows services to signal their readiness to the init system. Because systemd's notification library (libsystemd) links against liblzma, the patched sshd transitively inherits this dependency.

This distribution-specific patching decision was not a security flaw in itself. It was a routine integration choice made to improve systemd compatibility. However, it created the dependency chain that the backdoor required. Distributions that do not apply to this patch - including Arch Linux, Gentoo, and NixOS - are architecturally protected regardless of which XZ version they run.

The IFUNC (Indirect Function) mechanism exploited by the backdoor is a glibc extension that allows a function's implementation to be selected at library load time based on the runtime environment (for example, choosing between AVX2 and SSE2 implementations of a function based on CPU capabilities). The backdoor registers a malicious IFUNC resolver for

RSA_public_decrypt(), causing the function pointer to resolve to the attacker's code rather than the legitimate OpenSSL implementation. This hook is established at library load time and affects any process that loads the backdoored liblzma, including sshd.

2.3 The Build System: Autotools and the Role of Configure Scripts

XZ Utils, like many open-source projects, uses the GNU Autotools build system (autoconf/automake) to manage its compilation process. Autotools generates a **configure** shell script from template files (principally **configure.ac** and **m4/*.m4**) that, when executed, detects the host environment and produces a **Makefile** tailored to that environment. The configure script is typically lengthy, complex, and generated rather than hand-written, making it a poor candidate for close human scrutiny.

The critical distinction in this attack is between the project's **Git repository**, hosted on GitHub and publicly accessible and the **release tarballs**, compressed archives distributed to Linux distributions for packaging. These are not the same artifact. The Git repository contains the source files from which the configure script is generated. The release tarball contains the pre-generated configure script, additional build support files, and, in this case, malicious binary test files that served as the payload delivery mechanism.

The file modified to deliver the payload was **m4/build-to-host.m4**, a build support file present in the tarball but absent from the Git repository. This file contained an obfuscated shell script pipeline that, when executed during the configure phase, extracted the malicious payload from binary test files and injected it into the build process. Because the backdoor existed only in the tarball and not in the repository, standard open-source code review, which typically examines the repository, would not have detected it.

2.4 Threat Landscape Context

Supply chain attacks have emerged as one of the most consequential categories of cyber threat over the preceding decade. Their effectiveness derives from the exploitation of trust relationships: when an organisation trusts a software component, they implicitly trust everything that component depends on and everything involved in its production.

Attack	Year	Vector	Attribution	Impact
SolarWinds	2020	Build system compromise (Orion)	Unattributed	18,000+ victims, US government agencies
Log4Shell	2021	Dependency exploitation (Log4j)	N/A (vulnerability)	Hundreds of millions of vulnerable systems
Codecov	2021	CI/CD pipeline injection	Unknown	Hundreds of companies, credential theft
XZ Utils	2024	Build system + social engineering	Unattributed	Near-miss; caught before stable deployment

The XZ attack represents a qualitative advance over prior supply chain attacks in one critical dimension: the attack vector was not a technical vulnerability in the build infrastructure, but a human being, a fictitious identity that spent three years building genuine trust before exploiting it.

The attack maps to the following MITRE ATT&CK techniques: **T1195.001** (Compromise Software Dependencies and Development Tools), **T1554** (Compromise Client Software Binary), **T1585** (Establish Accounts), **T1562** (Impair Defenses), **T1078** (Valid Accounts), and **T1098** (Account Manipulation). A complete ATT&CK mapping is provided in Appendix B. ^{[25][26]}

3. What Did the Threat Actor Do?

3.1 Identity: JiaT75 / Jia Tan

The threat actor operated under the GitHub username **JiaT75** and the display name **Jia Tan**. The account was created on 26 January 2021 and remained dormant for several months before beginning a sustained pattern of open-source contribution. Over the course of approximately 2.6 years, the account authored over 450 commits across multiple projects, the substantial majority of which were entirely legitimate and technically sound.

The identity of the person or persons behind JiaT75 remains unknown. No government or intelligence agency has issued a formal public attribution. The JiaT75 account presents the characteristics of a carefully constructed fictitious persona: an aged account with a sustained contribution history and no personal information. The coordination of the JiaT75 account with multiple supporting sockpuppet identities strongly suggests the involvement of an organised team rather than an individual.

3.2 Phase 1: Account Creation and Initial Contributions (Jan 2021 – Nov 2022)

Following the account's creation in January 2021, JiaT75 began making contributions to open-source projects in early 2022. The contributions were technically competent, responsive to reviewer feedback, and genuinely useful. This pattern served two functions: building the reputation capital that would later be required to acquire maintainer status and establishing a plausible history that would withstand scrutiny.

One contribution during this phase warrants specific note: a patch to **libarchive** that replaced a call to **safe_fprintf()** with the standard but less safe **fprintf()**. This is the only contribution from this period that security researchers have identified as potentially malicious in intent. It was merged without issue, suggesting either that it passed review without concern or that its significance was not recognised at the time. ^[12]

3.3 Phase 2: Social Engineering Campaign (May – Dec 2022)

In May 2022, a coordinated pressure campaign began on the xz-devel mailing list. Multiple accounts, subsequently identified as sockpuppets, began posting complaints about the pace of XZ Utils development and the responsiveness of its sole maintainer, Lasse Collin.

The accounts identified as part of this campaign include Jigar Kumar, Dennis Ens, krygorin4545, and misoeater91. All were created after 2021. Their messages expressed frustration with slow release cycles, questioned the project's maintenance quality, and consistently advocated for Jia Tan as a capable contributor who should be given additional responsibility.

```
# xz-devel mailing list excerpt, ~May 2022
From: Jigar Kumar <jigar.kumar@[redacted]>

"XZ Utils has been stagnating. Jia Tan has shown they are capable
and responsive. It would benefit the project to give them a larger
role. Lasse, you seem overwhelmed."
# - Jigar Kumar (fictitious identity)
```

Lasse Collin, the sole maintainer of XZ Utils, was experiencing personal difficulties during this period and had communicated to the community that the project might see slower progress. The sockpuppet campaign exploited this vulnerability precisely: it manufactured apparent community pressure to delegate at the moment when Collin was least positioned to resist it.

WARNING

The sockpuppet campaign represents a form of social engineering that has no direct technical mitigation. The attack exploited the gap between the social norms of open source communities, which are built on trust and goodwill, and the adversarial reality faced by critical infrastructure maintainers. This is an IAM problem: the 'identity' of a project contributor is verified only by their track record, which can be fabricated.

3.4 Phase 3: Position of Trust (Dec 2022 - Dec 2023)

By December 2022, JiaT75 had been granted co-maintainer status on XZ Utils. In the same month, the project's hosting was migrated from SourceForge to GitHub, with JiaT75 creating the GitHub organisation (tukaani-project) and establishing their email address as the project's default contact. This migration gave JiaT75 organisational administrator access to the project's GitHub presence, including the ability to create and sign official releases.

In July 2023, JiaT75 submitted a pull request to the **google/oss-fuzz** repository, a Google-operated automated fuzzing service for open-source projects, specifically disabling IFUNC support in the fuzzing configuration for XZ Utils. IFUNC is the precise glibc mechanism that the backdoor would later exploit. Disabling this fuzzing path prevented oss-fuzz from detecting anomalous behaviour in IFUNC resolution, a preparatory countermeasure executed months before the payload was inserted. This is one of the clearest indicators of premeditation in the entire operation. ^[12]

During 2023, JiaT75 made their first solo release of XZ Utils as a co-maintainer, further establishing the legitimacy of their release-signing key and creating the trust precedent that would be exploited in the 2024 releases.

3.5 Phase 4: Payload Insertion (Feb - Mar 2024)

The operational phase of the attack compressed three years of preparation into approximately ten weeks.

Date	Action
23 Feb 2024	First malicious commit: binary payload files added disguised as test data
26 Feb 2024	Landlock security feature sabotaged in a follow-up commit
24 Feb 2024	XZ Utils 5.6.0 released - first backdoored version
9 Mar 2024	XZ Utils 5.6.1 released - reliability fixes to the backdoor
25 Mar 2024	Fake account 'Hans Jansen' pressures Debian to fast-track adoption (Debian bug #1067708)
27 Mar 2024	Andres Freund contacts Debian security team about SSH anomalies
29 Mar 2024	Freund publishes full disclosure to oss-security mailing list

The release of XZ 5.6.1 on 9 March 2024 - issued just 13 days after 5.6.0 - is a notable OPSEC failure. The stated purpose was to fix unspecified bugs, but it was subsequently determined that the update addressed reliability issues with the backdoor itself: certain hook failures were causing detectable crashes. Issuing a rapid point release to fix a non-public defect in a stable utility is anomalous behaviour and, in retrospect, a detectable signal. ^[7]

4. How Was It Done?

4.1 Payload Delivery: Build System Injection

The backdoor was not introduced into XZ Utils' publicly visible source code. It was introduced into the **release tarball**, the compressed archive distributed to Linux distributions for packaging, via a modified Autotools build support file. The specific file modified was **m4/build-to-host.m4**, which is present in the tarball but absent from the GitHub repository.

This architectural distinction - tarball versus repository - is the foundation of the attack's stealth. When security researchers or Linux distribution maintainers review an open-source project, they typically review the source repository. The release tarball is typically assumed to be a faithful product of that repository. In this case, it was not.

```
# Tarball vs. Git repository - the critical divergence

TARBALL: xz-5.6.0.tar.gz
m4/build-to-host.m4      <- MALICIOUS (absent from git)
tests/files/bad-3-corrupt_lzma2.xz  <- PAYLOAD
tests/files/good-large_compressed.lzma <- PAYLOAD

GIT:    github.com/tukaani-project/xz
m4/build-to-host.m4      <- CLEAN (does not exist in git)
tests/files/bad-3-corrupt_lzma2.xz  <- CLEAN
tests/files/good-large_compressed.lzma <- CLEAN
```

4.2 The Three-Stage Injection Process

The payload delivery operates in three sequential stages, each building on the previous.

Stage 1: Configure Script Execution

The modified **build-to-host.m4** causes the Autotools-generated **configure** script to execute a `sed/tr/xz` pipeline at the end of the configure phase. This pipeline reads the binary test files, decodes them, and passes the result to **/bin/sh** for execution. The pipeline is obfuscated using character substitution to avoid simple pattern matching.

```
# Malicious pipeline in generated configure script
# (present only when built from tarball, not git)

am__test_x='sed s/a/x/g'
| tr "\t _" " \t_"
| xz -d
| /bin/sh >&6 2>&1

# Effect: extracts and executes hidden shell script
# from binary test files at build time
```

Stage 2: Object File Extraction

The shell script executed in Stage 1 decodes a prebuilt compiled object file (.o) using RC4 stream cipher decryption. The object file is embedded within the binary test files disguised as corrupt or legitimate lzma data. Once decoded, the object file is placed in a location where it will be picked up by the linker during the compilation of liblzma.

```
# Stage 2: RC4-decoded object file extraction
# The string 'yolAbejyiejuvnup=' is a base64-encoded RC4 key

eval $(echo 'yolAbejyiejuvnup=' | ...)
# Decodes payload.o from binary test files
# Links into liblzma during compilation
```

Stage 3: IFUNC Resolver Registration

The compiled object file, once linked into liblzma, registers IFUNC resolvers for target functions using GCC's `__attribute__((ifunc("resolver")))` mechanism. IFUNC (Indirect Function) is a glibc and GCC feature that allows a function to be resolved to different implementations at library load time, based on the runtime environment. Legitimate uses include CPU feature detection (e.g., selecting between AVX2 and SSE2 code paths). The backdoor abuses this mechanism to substitute malicious implementations of cryptographic functions.

```
# Stage 3: IFUNC hook registration
# The malicious .o file contains:

__attribute__((ifunc("resolve_RSA_public_decrypt")))
int RSA_public_decrypt(...);

static void *resolve_RSA_public_decrypt(void) {
    // Returns malicious implementation if conditions met
    // Returns legitimate implementation otherwise
}
```

4.3 The Backdoor Hook: RSA_public_decrypt()

The primary target of the IFUNC hook is `RSA_public_decrypt()`, an OpenSSL function called during SSH public key authentication. When `sshd` processes an incoming SSH connection, it invokes this function to validate the client's public key signature. The backdoored version of this function, substituted via the IFUNC resolver, performs the following logic:

```
# Simplified pseudocode of the malicious RSA_public_decrypt hook

int hooked_RSA_public_decrypt(int flen, unsigned char *from,
                             unsigned char *to, RSA *rsa, int padding) {
    // Extract command number from RSA modulus N bytes 0-15
    cmd = N[0..15] % 4;

    // Verify Ed448 signature on the RSA key payload
    if (verify_ed448_signature(N, attacker_public_key)) {
        switch (cmd) {
```



```
        case 0x01: return bypass_authentication();
        case 0x02: return execute_shell_command(payload);
        case 0x03: return execute_with_uid_gid(payload);
    }
}
// No match: invoke legitimate function, no trace left
return real_RSA_public_decrypt(flen, from, to, rsa, padding);
}
```

The hook uses the **RSA modulus N** (the public key material) as a covert channel: bytes 0-15 of N encode a command number, and the key is signed with the attacker's **Ed448** private key. Ed448 is an elliptic curve signature scheme; verification uses the corresponding public key, which is embedded in the malicious object file. Only connections presenting a key signed by the attacker's private key trigger the backdoor behaviour. All other connections proceed through the legitimate authentication path with no detectable deviation.

4.4 The Five Exploitation Conditions

The backdoor required all five of the following conditions to be simultaneously satisfied on a target system:

Condition	Requirement	Reason
1	Linux x86-64 architecture	The malicious object file was compiled for x86-64; it would not load on ARM or other architectures
2	glibc (not musl)	IFUNC resolver support is a glibc extension; musl libc does not support IFUNC
3	XZ Utils 5.6.0 or 5.6.1	Only these versions contain the backdoor; all other versions are unaffected
4	OpenSSH linked against libsystemd (sd_notify patch)	Creates the liblzma dependency in sshd; absent on distributions that do not apply this patch
5	SSH daemon running and accessible	sshd must be active and listening for the hook to be triggerable

The specificity of these conditions was deliberate: it targeted the maximum-value attack surface (production Linux servers) while minimising exposure to detection by limiting activation to systems meeting a precise profile.

4.5 Evasion Techniques

The backdoor incorporated multiple layers of evasion:

- Payload concealed in binary test files rather than plain-text source code, bypassing text-based code review.
- Absent from the GitHub repository, only present in release tarballs, avoiding scrutiny of the public repository.
- Multi-stage obfuscation: character substitution, base64, RC4, and indirect function registration.

- Dormant unless triggered by a specific Ed448-signed payload, no beaconing, no anomalous traffic.
- No log entries generated when the backdoor activates or when authentication is bypassed.
- oss-fuzz IFUNC fuzzing pre-disabled months in advance to prevent automated detection.
- The rapid 5.6.1 release to fix backdoor reliability issues was an OPSEC error. It is anomalous to release a point update for a stable utility to fix a non-public defect, but was not investigated until after the broader discovery. ^[8]

4.6 Tool-Based Configuration Analysis

The **CVE-2024-3094 Research Toolkit** was applied to three representative system profiles to demonstrate the vulnerability conditions in practice. Each profile was evaluated against the five exploitation conditions described in Section 4.4. The tool accepts output from the following three commands:

```
# Command 1: XZ Utils version
$ xz --version

# Command 2: sshd dynamic library dependencies
$ ldd /usr/sbin/sshd

# Command 3: SSH daemon status
$ systemctl is-active ssh
```

Profile A: Fully Vulnerable (Debian Sid, XZ 5.6.1)

All five conditions are satisfied. The tool correctly identifies the system as vulnerable, flags the backdoored XZ version, confirms liblzma and libsystemd in the sshd dependency chain, confirms glibc, and confirms an active SSH daemon. The dependency chain visualiser renders the complete attack path in its compromised state.

The screenshot displays the 'System Vulnerability Checker' interface. On the left, under 'COMMANDS TO RUN', three commands are listed with their corresponding outputs: 'xz --version' (XZ Utils 5.6.1), 'ldd /usr/sbin/sshd' (showing liblzma and libsystemd in the dependency chain), and 'systemctl is-active ssh' (active (running)). The main panel shows a 'VERDICT' of 'VULNERABLE' in red. Below this, a 'CONDITION BREAKDOWN' section lists five conditions, all marked as 'FAIL': 'XZ Utils version' (5.6.1 is backdoored), 'liblzma linked into sshd' (liblzma is present in the dynamic library chain), 'libsystemd linked into sshd' (libsystemd is present), 'glibc present (IFUNC support)' (glibc detected), and 'SSH daemon running' (sshd is active). A 'VISUALISE THIS SYSTEM'S DEPENDENCY CHAIN' button is visible above the breakdown. At the bottom, there are 'ANALYSE SYSTEM' and 'RESET' buttons.

Profile B: Safe by Version (Ubuntu LTS, XZ 5.4.6)

The XZ version condition fails at the first check (5.4.6 is not affected). Additionally, neither liblzma nor libsystemd appear in the sshd dependency chain, reflecting Ubuntu LTS's packaging decisions. The tool correctly identifies the system as not vulnerable and the visualiser shows the liblzma and libsystemd nodes dimmed, indicating their absence from the dependency chain.

System Vulnerability Checker
Run the three commands on your target system and paste the output. The checker evaluates whether that system satisfies all five conditions for CVE-2024-3094 exploitation.

COMMANDS TO RUN

- XZ version: `xz --version`
- sshd dependencies: `ldd /usr/sbin/sshd`
- SSH daemon status: `systemctl is-active ssh`

1. XZ VERSION (xz --version)

```
xz (XZ Utils) 5.4.6
liblzma 5.4.6
```

2. SSHD LDD OUTPUT (ldd /usr/sbin/sshd)

```
linux-vdso.so.1 (0x00007ffc8d9f2000)
libcrypto.so.3 => /lib/x86_64-linux-gnu/libcrypto.so.3 (0x00007fb1c0000000)
libpam.so.0 => /lib/x86_64-linux-gnu/libpam.so.0 (0x00007fb1c3c00000)
libssl.so.1 => /lib/x86_64-linux-gnu/libssl.so.1 (0x00007fb1c3d00000)
```

3. SSH STATUS (systemctl is-active ssh)

```
active (running) since Thu 2024-03-28 09:15:33 UTC; 1 day 4h ago
```

VERDICT: NOT VULNERABLE
This system does not satisfy all conditions for exploitation. At least one critical dependency in the attack chain is absent.

CONDITION BREAKDOWN

- XZ Utils version** (PASS): Version 5.4.6 is not affected. Only 5.6.0 and 5.6.1 contain the backdoor.
- liblzma linked into sshd** (PASS): liblzma is NOT linked into sshd. The hook has no injection path. System not affected via this vector.
- libsystemd linked into sshd** (PASS): libsystemd NOT found. This distro does not patch OpenSSH with sd_notify — libzma linkage is absent.
- glibc present (IFUNC support)** (FAIL): glibc detected. IFUNC resolver mechanism available — required for the backdoor's function hook.
- SSH daemon running** (FAIL): sshd is active. A running SSH daemon with the backdoored liblzma loaded is the final exploitation condition.

Profile C: Architecturally Protected (Arch Linux, XZ 5.6.0)

This profile represents the most analytically instructive case. XZ Utils 5.6.0 is installed, so the version condition fails. However, the attack chain additionally breaks at libsystemd: Arch Linux does not apply the sd_notify patch to OpenSSH, meaning libsystemd is not linked into sshd, and consequently liblzma is not present in the sshd dependency chain. The system is not vulnerable despite running the backdoored XZ version.

System Vulnerability Checker
Run the three commands on your target system and paste the output. The checker evaluates whether that system satisfies all five conditions for CVE-2024-3094 exploitation.

COMMANDS TO RUN

- XZ version: `xz --version`
- sshd dependencies: `ldd /usr/sbin/sshd`
- SSH daemon status: `systemctl is-active ssh`

1. XZ VERSION (xz --version)

```
xz (XZ Utils) 5.6.0
liblzma 5.6.0
```

2. SSHD LDD OUTPUT (ldd /usr/sbin/sshd)

```
linux-vdso.so.1 (0x00007ffc8d9f2000)
libcrypto.so.3 => /usr/lib/libcrypto.so.3 (0x00007fb2c0000000)
libpam.so.0 => /usr/lib/libpam.so.0 (0x00007fb2c3c00000)
libssl.so.1 => /usr/lib/libssl.so.1 (0x00007fb2c3d00000)
```

3. SSH STATUS (systemctl is-active ssh)

```
active (running) since Sat 2024-03-30 08:22:11 UTC; 45min ago
```

VERDICT: NOT VULNERABLE
This system does not satisfy all conditions for exploitation. At least one critical dependency in the attack chain is absent.

CONDITION BREAKDOWN

- XZ Utils version** (FAIL): Version 5.6.0 is backdoored (5.6.0 and 5.6.1 contain the malicious payload).
- liblzma linked into sshd** (PASS): liblzma is NOT linked into sshd. The hook has no injection path. System not affected via this vector.
- libsystemd linked into sshd** (PASS): libsystemd NOT found. This distro does not patch OpenSSH with sd_notify — libzma linkage is absent.
- glibc present (IFUNC support)** (FAIL): glibc detected. IFUNC resolver mechanism available — required for the backdoor's function hook.
- SSH daemon running** (FAIL): sshd is active. A running SSH daemon with the backdoored liblzma loaded is the final exploitation condition.

Key Finding

Profile C demonstrates that vulnerability to CVE-2024-3094 was architecture-specific, not merely version-specific. A system administrator running XZ 5.6.0 on Arch Linux would not have been exposed to exploitation, while the identical XZ version on Debian Sid created a fully exploitable attack surface. This finding has implications for how Linux distribution design choices affect the attack surface of their downstream systems.

5. Why Was It Done?

5.1 Nature of the Backdoor: Espionage, Not Crime

The design characteristics of the backdoor are inconsistent with financially motivated cybercrime and consistent with state intelligence objectives. Several features support this assessment:

- **Key-gated access:** The backdoor responds only to connections presenting a payload signed by the attacker's Ed448 private key. It is not exploitable by third parties, cannot be used for mass criminal operations, and cannot be resold or leaked for general use without the key.
- **No financial mechanism:** The payload provides unauthenticated shell access. There is no ransomware component, no cryptocurrency wallet, no data exfiltration trigger. The capability provided is persistent, silent access - the signature of intelligence collection.
- **Forensic silence:** The hook generates no log entries and produces no detectable network traffic. A financially motivated actor typically seeks rapid exploitation; a state-intelligence actor seeks long-term, undetected access.
- **Target selection:** The specific exploitation conditions targeted production Linux servers, government systems, financial infrastructure, telecommunications, rather than consumer endpoints or Windows systems targeted by criminal ransomware.

5.2 Technical Attribution Evidence

Several technical indicators provide partial attribution evidence:

- **Coordinated identity management:** The use of multiple sockpuppet accounts, all created after 2021 and coordinated in their messaging, indicates an organised team operation rather than an individual actor.
- **Technical sophistication:** The multi-stage obfuscation, the IFUNC mechanism exploitation, and the preemptive disabling of oss-fuzz detection are collectively at a level of sophistication assessed by multiple independent security researchers as nation-state tier. ^{[9][10]}
- **Operational patience:** The three-year timeline from account creation to payload insertion is inconsistent with criminal operations, which prioritise speed. It is consistent with intelligence operations, which prioritise access quality and deniability.

5.3 Strategic Target Assessment

The target of the backdoor, SSH infrastructure on Linux servers, represents access to the administrative backbone of global internet infrastructure. Governments, financial institutions, telecommunications providers, and defense contractors all rely on Linux servers managed via SSH. Persistent, silent access to this infrastructure at scale would provide an intelligence capability of extraordinary value.

The operation bears comparison with **SolarWinds (2020)**: in both cases, a trusted software distribution mechanism was compromised to deliver a payload to a large number of downstream targets; in both cases, the payload was designed for covert, persistent access

rather than immediate exploitation; and in both cases, the operation required significant advance planning and inside access. ^[7]

5.4 Current Attribution Status

As of the time of writing, no government has issued an official public attribution for CVE-2024-3094. The community consensus, reflected in analyses by Kaspersky, JFrog, Datadog, and others, is that the operation was state-sponsored. The absence of official attribution reflects both the inherent difficulty of supply chain attack attribution and the intelligence community's preference for not disclosing collection methods.

The OPSEC failures identified in the operation, most notably the rapid 5.6.1 release to fix reliability issues, are consistent with an operation under time pressure, possibly because the operator assessed that the window for undetected deployment was closing. The pressure campaign on Debian maintainers in late March 2024 supports this interpretation. ^{[14][15]}

6. What Would Have Happened?

6.1 Scope of Exposure at Time of Discovery

At the time of Freund's disclosure on 29 March 2024, the backdoored versions of XZ Utils had been available for approximately five weeks (5.6.0 since 24 February) and had been incorporated into the following distributions:

Distribution	Version	Status	Protection Type
Debian unstable/testing/experimental	5.6.0, 5.6.1	VULNERABLE	N/A
Fedora Linux 40 beta	5.6.0	VULNERABLE	N/A
Fedora Rawhide	5.6.0, 5.6.1	VULNERABLE	N/A
Kali Linux (images 26–28 Mar 2024)	5.6.0	VULNERABLE	N/A
openSUSE Tumbleweed (specific dates)	5.6.0, 5.6.1	VULNERABLE	N/A
Debian Stable	5.4.x	NOT AFFECTED	Version
Ubuntu LTS	5.4.x	NOT AFFECTED	Version
Red Hat Enterprise Linux	5.4.x	NOT AFFECTED	Version
Amazon Linux	5.4.x	NOT AFFECTED	Version
Arch Linux	5.6.0 (any)	NOT AFFECTED	Architecture (no sd_notify)
Gentoo	Any	NOT AFFECTED	Architecture (no sd_notify)
NixOS	Any	NOT AFFECTED	Architecture (no sd_notify)
Alpine / musl-based	Any	NOT AFFECTED	Architecture (no glibc/IFUNC)

The backdoor was caught at the pre-release gate. The distributions that were affected - Debian unstable, Fedora Rawhide, Fedora 40 beta - are development and pre-release channels not recommended for production use. The stable, enterprise-grade distributions used by the vast majority of production servers were not affected.

However, this should not be interpreted as evidence that the attack's potential was limited. Development and testing distributions are frequently run on servers used by developers, researchers, and security professionals - high-value targets for intelligence collection.

Furthermore, the attack was caught with approximately five weeks remaining before the stable release cycle would have incorporated the backdoored versions.

6.2 Attack Capability If Deployed

Had the backdoor reached stable production distributions and gone undetected for a period of active use, the attacker would have possessed the following capabilities on any affected system with port 22 accessible:

- **Unauthenticated root access:** Authentication is entirely bypassed. No credentials, no keys, no MFA token is required.
- **Arbitrary command execution:** Command numbers 0x02 and 0x03 provide the ability to execute arbitrary shell commands, including as a specified UID/GID.
- **Forensic silence:** No log entries are generated by the hook. Standard SSH access logging, SIEM alerts based on authentication events, and audit trails would show nothing.
- **Persistence capability:** Post-exploitation, the attacker could install additional implants, create backdoor accounts, or exfiltrate data - using standard techniques that would be indistinguishable from legitimate administrator activity.
- **Scale:** The same key would work on every affected system. A single operator with the private key could access millions of servers worldwide without any system-specific preparation.

6.3 Systemic Impact Scenarios

Three scenarios illustrate the range of potential consequences had the attack succeeded and gone undetected for a significant period.

Scenario A: Targeted Espionage (Assessed Most Likely)

Consistent with the backdoor's design and the presumed intent of its operator, targeted espionage involves selective, covert use against high-value servers: government systems, defence networks, financial institutions, and telecommunications infrastructure. Access would be used for intelligence collection rather than disruption. The operator would maintain access silently for an extended period, extracting information while avoiding detection. This scenario is consistent with the intelligence tradecraft evident throughout the operation.

Scenario B: Infrastructure Disruption

The same access capability that supports espionage also supports disruption. Simultaneous activation of the backdoor against critical infrastructure, power grid management systems, financial settlement infrastructure, emergency services coordination, could cause widespread disruption. The Linux footprint in these sectors is substantial. This scenario is assessed as less likely as an initial use case, given the intelligence value of persistent silent access, but represents the worst-case potential of the capability.

Scenario C: Key Compromise

The key-gated design assumes the Ed448 private key remains under the exclusive control of its creator. If the key were obtained by a third party - through espionage against the operation's operators, through defection, or through operational failure - the backdoor would become available to additional actors. Criminal ransomware deployment at scale against affected Linux servers would become possible.

6.4 Cascading Supply Chain Effects

The exposure window was not limited to direct users of the affected distributions. Docker Hub images based on Debian or Fedora builds from the exposure period would have incorporated the backdoored liblzma. Binarly, a firmware and supply chain security company, identified **affected Docker images as late as August 2025**, more than a year after the initial disclosure, indicating the persistence of vulnerable artifacts in container registries even after the primary packages were patched. CI/CD pipelines pulling these base images would have built downstream software against the compromised library, creating a second tier of exposure.

7. Identity and Access Management Implications

7.1 The Failure Mode This Attack Represents

Identity and Access Management is built on a foundational assumption: that the authentication mechanism is trustworthy. IAM programmes invest in strong credentials, multi-factor authentication, privileged access controls, and Zero Trust architectures, all of which are designed to govern who can access what. All these controls, without exception, assume that the underlying authentication mechanism correctly reports whether a presented credential is valid.

CVE-2024-3094 invalidated this assumption. The backdoor did not steal credentials, did not exploit a misconfiguration, and did not bypass access controls in the conventional sense. It **replaced the authentication function itself** - substituting the attacker's code for the legitimate `RSA_public_decrypt()` implementation - so that the authentication system would report any connection from the attacker as successfully authenticated, regardless of what credentials were presented.

This is a qualitatively different threat model. It is not a failure of IAM policy, credential management, or access governance. It is an integrity failure at the layer below IAM - a compromise of the mechanism that IAM depends on. Understanding this distinction is essential for IAM practitioners assessing what this attack implies for their programmes.

7.2 The Attack Chain as a Layered IAM Problem

The attack can be modelled as a series of identity and trust failures at successive layers:

Layer	Trust Assumed	How It Was Exploited
Contributor identity	Open-source contributors are who they claim to be	JiaT75 was a fictitious persona with fabricated history
Maintainer role	Maintainers are vetted and accountable	Co-maintainer status granted based on fabricated reputation
Release integrity	Official releases faithfully represent the source repository	Release tarball diverged from repository; malicious code absent from repo
Code signing	GPG-signed releases prove authenticity	Attacker signed releases with their own (attacker-controlled) key
Build process	Build tools produce predictable output from source	Build system modified to inject payload during compilation
Authentication stack	sshd correctly validates public key signatures	<code>RSA_public_decrypt()</code> replaced with malicious implementation

Each layer represents a trust relationship that was exploited. Traditional IAM addresses the topmost layers (identity, role, access policy) but has no mechanisms addressing the lower layers (build process integrity, authentication stack integrity).

7.3 Software Supply Chain as an Identity Problem

The open-source contribution model is, at its core, an identity system. When a patch is merged into a project, the implicit claim is that the contributor is a trustworthy member of the community who has acted in good faith. When a release is signed with a maintainer's GPG key, the implicit claim is that the maintainer is who they say they are and that the release faithfully represents the project's source.

These claims are verified only by track record and reputation - the digital equivalent of 'we know them, they've always been reliable.' There is no background verification, no credential issuance, no formal identity proofing, no privileged access review. The onboarding process for an open-source maintainer consists entirely of demonstrating competence over time, which is precisely what JiaT75 did, systematically and over three years.

For IAM practitioners, this maps directly to the insider threat model. The threat actor performed the equivalent of a long-tenure employee who was never subject to periodic access review, whose role escalation was approved informally, and who exploited their privileged position to introduce a backdoor into a production system. The difference is that the 'organisation' in question had no IAM programme at all.

7.4 Privileged Access Management Analysis

JiaT75's acquisition of co-maintainer status and the ability to create and sign official releases represents a privileged access escalation with no procedural controls. In an enterprise context, release signing authority is analogous to production deployment access, it should require formal approval, peer review, and periodic recertification.

The backdoor itself, once in place, granted its operator root-equivalent access to affected systems, bypassing the entire PAM stack. Controls including sudo policy, RBAC, SELinux, and AppArmor all operate above the layer at which the hook was inserted. An attacker entering via the backdoor would have the same privileges as the sshd process and would be able to escalate from there using standard post-exploitation techniques indistinguishable from legitimate administrator activity.

Just-in-Time (JIT) privileged access management, which limits the duration of privileged sessions, would not have prevented exploitation: the backdoor grants access on demand, so there is no standing privilege to time-limit.

7.5 IAM Control Effectiveness Analysis

The following table maps common IAM and security controls to their effectiveness against CVE-2024-3094, distinguishing between prevention, detection, and containment.

Control	Effectiveness	Rationale
Multi-Factor Authentication (MFA)	NONE	Hook fires before auth logic; MFA is never reached
PAM (Pluggable Authentication Modules)	NONE	Operates above the hooked layer; bypassed
SSH Key Management and Rotation	NONE	Key validity irrelevant; hook bypasses the check entirely
Zero Trust Network Access	NONE	Authentication itself is compromised; ZTNA cannot compensate
RBAC / Least Privilege (user level)	NONE	Access is granted pre-authorisation; RBAC is never consulted
Code Signing (GPG)	NONE	Attacker signed releases with their own trusted key; signing proves identity of signer, not integrity of code
SBOM / Dependency Inventory	PARTIAL	Identifies component presence; does not detect malicious content within a component
SELinux / AppArmor	PARTIAL	Can restrict post-exploitation lateral movement; does not prevent the hook or initial access
Multi-party Release Approval	HIGH	Would require a second maintainer to independently verify the tarball matched the repository
Tarball vs. Git Repository Diff Check	HIGH	Would detect the divergence between the release tarball and the GitHub source
Runtime Binary Integrity (IMA/dm-verity)	HIGH	Would detect the modified liblzma.so at load time before the hook executes
Anomaly Detection / Performance Monitoring	HIGH	The 500ms SSH latency increase was the actual detection method; systematic monitoring would have found this sooner
Segregation of Duties (Commit vs. Release)	HIGH	Separating commit rights from release authority would have prevented a single actor from both introducing and shipping the payload
Maintainer Identity Verification	HIGH	Formal identity proofing before privilege escalation would have exposed the fictitious identity

IAM Key Finding

Every control rated NONE shares a common characteristic: it assumes the integrity of the authentication mechanism. Every control rated HIGH operates at or below the layer the attack targeted - the build process and the authentication stack. This finding suggests that IAM programmes must extend their scope downward, into software supply chain integrity and authentication stack integrity, to address this class of attack.

7.6 Implications for IAM Practitioners

The following recommendations are directed at IAM consultants and practitioners, particularly those advising organisations with significant Linux infrastructure:

Software Supply Chain as IAM Scope

The dependencies that an organisation's systems load at runtime should be treated as an identity question: who vouched for this code, and is that vouching verifiable? Software Bills of Materials (SBOMs) are a necessary but insufficient starting point, they identify what is present, not whether what is present is trustworthy. SBOM programmes should be extended to include integrity verification (hash comparison against known-good builds) for all critical dependencies.

Authentication Stack Integrity

IAM programmes should include authentication stack integrity monitoring as a control category. Runtime binary integrity checking (Linux Integrity Measurement Architecture, or IMA) verifies that loaded libraries match their expected cryptographic hashes before execution. This control would have detected the modified liblzma before the hook could execute.

Anomaly Detection as an IAM Control

The 500ms SSH latency increase that led to Freund's discovery was a performance anomaly, not a security alert. Systematic SSH latency monitoring, sshd CPU consumption tracking, and automated comparison of expected versus observed authentication timing would constitute an IAM-adjacent control that could detect this class of attack.

Vendor and Dependency Risk Management

Open source dependencies should be subject to supply chain risk management equivalent to that applied to commercial vendors under **NIST SP 800-161r1**.^[24] This includes identifying the maintainers of critical dependencies, assessing their organisational resilience (including bus factor), and monitoring for changes in maintainer composition.

7.7 Regulatory and Compliance Dimensions

The **EU Cyber Resilience Act (CRA)**, scheduled for enforcement by December 2027, imposes security requirements on products with digital elements and creates obligations for organisations that profit from open-source software.^[27] The XZ incident illustrates precisely the category of risk the CRA is designed to address: a critical open-source component maintained by a single volunteer with no institutional security programme.

Under **NIST SP 800-161r1**, the dependency on XZ Utils would be classified as a third-party software component requiring supply chain risk assessment. The controls that would have been most relevant, software integrity verification, maintainer vetting, and multi-party release approval, are directly addressed in NIST's supply chain security framework.

For organisations operating under SOC 2 or ISO 27001, the attack highlights gaps in how these frameworks address software supply chain risk. Both frameworks require vendor risk management, but neither provides specific guidance on open source dependencies or build system integrity. Security practitioners advising on compliance should consider explicitly extending vendor risk processes to critical open source components.

8. Community and Institutional Response

8.1 Immediate Technical Response (March – April 2024)

The response to Freund's disclosure was rapid and globally coordinated, demonstrating one of the genuine strengths of the open source security ecosystem: the ability to mobilise a large, distributed community around a shared threat.

- **Debian:** Immediately rolled back XZ Utils to version 5.4.6-0.2 across unstable, testing, and experimental branches.
- **Red Hat:** Issued emergency security advisory RHSA-2024:1618 within hours. Confirmed that Red Hat Enterprise Linux stable releases were not affected.^[18]
- **CISA:** Issued a public alert recommending immediate downgrade to XZ Utils 5.4.6 and published a detailed lessons-learned analysis.^[3]
- **Kali Linux:** Identified affected images (26–28 March 2024) and issued specific guidance.
- **Arch Linux, openSUSE:** Issued advisories and coordinated rollbacks.
- **GitHub:** Suspended the XZ Utils repository to prevent further access during the investigation.
- **OpenSSF:** The Open Source Security Foundation issued a joint alert with the OpenJS Foundation warning that the XZ incident may represent part of a broader pattern of social engineering attacks targeting critical open source projects.^[20]

The speed of the response - with patches, advisories, and rollbacks issued within hours of disclosure - reflects the maturity of the open-source security response infrastructure. However, the community was also clear that this speed was possible only because the attack was caught before reaching stable production distributions. A post-deployment response would have been orders of magnitude more complex.

8.2 Medium-Term Reforms (2024–2025)

The XZ incident catalysed a range of institutional responses:

- The Alpha-Omega Project, an OpenSSF initiative funding security work on critical open-source projects, expanded its scope and funding following the incident.
- Increased scrutiny of new maintainer applications became an informal norm across several major open-source projects.
- Discussion of reproducible builds, ensuring that release tarballs can be verified to be faithful products of the source repository, gained renewed urgency.
- CISA's 'Secure by Design' initiative incorporated lessons from the XZ incident into its guidance for software producers.
- The incident was cited in Congressional testimony in the United States as evidence for increased government investment in open-source security infrastructure.

8.3 What Has Not Been Fixed

Despite the response, the structural conditions that enabled the attack remain intact:

- **Maintainer burnout:** Critical open-source infrastructure continues to be maintained by volunteers with no institutional support. The bus factor of one that made XZ Utils vulnerable exists in hundreds of other projects.
- **No standardised vetting process:** There is no industry-wide standard for vetting new open-source maintainers before granting release authority.
- **Binary files in repositories:** Binary test files, prebuilt objects, and other non-text assets in source repositories remain a detection blind spot for code review processes.
- **Tarball versus repository divergence:** The architectural possibility that release tarballs may differ from repository content remains. Reproducible builds address this but have not been universally adopted.
- **Funding:** The fundamental economic problem - that organisations deriving significant commercial value from open source infrastructure contribute little to its security - has not been resolved, though regulatory pressure (particularly from the EU CRA) is beginning to change incentive structures.

9. Conclusions

9.1 Summary of Findings

What the Threat Actor Did

A fictitious identity (JiaT75 / Jia Tan) conducted a multi-phase supply chain attack spanning approximately three years. The operation involved identity fabrication, sustained legitimate contribution to build reputation capital, coordinated social engineering using multiple sockpuppet accounts to manufacture community pressure, acquisition of co-maintainer status and release signing authority, and the insertion of a malicious payload into official release tarballs in January-February 2024.

How the Attack Was Technically Executed

The backdoor was delivered via a modified Autotools build support file (**m4/build-to-host.m4**) present only in release tarballs, not in the GitHub repository. The payload - concealed in binary test files - was extracted by a sed/tr/xz pipeline during the configure phase and injected as a prebuilt object file into the liblzma compilation. The object file registered glibc IFUNC resolvers that replaced **RSA_public_decrypt()** with a malicious implementation, enabling unauthenticated command execution on any system loading the backdoored liblzma where sshd was linked against libsystemd.

Why the Attack Was Carried Out

The backdoor's design - key-gated, forensically silent, providing persistent shell access rather than immediate financial extraction - is consistent with state intelligence objectives.

What Would Have Happened

Had the attack reached stable production distributions and remained undetected, the operator would have possessed unauthenticated root access, via a single private key, to every affected server worldwide with SSH exposed. The most likely use case was targeted espionage against high-value infrastructure. The attack was detected approximately five weeks before the stable release window and before any confirmed exploitation of production systems.

9.2 IAM-Specific Conclusions

The attack represents a threat model that falls outside the scope of conventional IAM frameworks. The following conclusions are offered for IAM practitioners:

1. **Authentication stack integrity must be an IAM concern.** The assumption that the authentication mechanism is trustworthy must be treated as a security control, not an axiom. Runtime binary integrity verification (IMA/dm-verity) should be part of the security baseline for organisations relying on SSH for privileged access.
2. **Software supply chain trust is an identity problem.** The question 'who vouches for this code?' must be answered with the same rigour as 'who is this user?' SBOM programmes should be extended to include integrity verification, not just inventory.
3. **Open-source maintainer risk is a vendor risk.** Organisations depending on open source components with a bus factor of one, no institutional security programme, and no formal maintainer vetting process are exposed to a risk category not addressed by conventional vendor management frameworks.

4. **No current IAM control would have prevented exploitation.** MFA, PAM, Zero Trust, RBAC, and code signing all failed against this attack because they operate above the layer that was compromised. This is not an indictment of these controls but an indication that they must be complemented by controls at lower layers.
5. **Anomaly detection is undervalued as an IAM control.** The actual detection mechanism - a 500ms SSH latency anomaly noticed by a single engineer - was not a security control at all. Systematic performance monitoring of authentication services should be treated as an IAM-adjacent detection control.

9.3 Tool Contribution

The **CVE-2024-3094 Research Toolkit** developed as part of this study provides a reproducible analytical framework for evaluating system exposure to this specific vulnerability. The tool demonstrated that vulnerability was simultaneously version-specific (XZ 5.6.0/5.6.1) and architecture-specific (glibc + systemd OpenSSH patch required).

The analysis of Profile C (Arch Linux with XZ 5.6.0) is the tool's most significant finding: a system running the backdoored XZ version was not exploitable due to an architectural difference in how its distribution packages OpenSSH. This finding has implications for how Linux distribution design choices — specifically the decision whether to apply the `sd_notify` patch to OpenSSH — affect the attack surface of their downstream systems, and argues for a more explicit treatment of packaging decisions as security decisions.

9.4 Future Work

The following areas warrant further research:

- Systematic auditing of binary test files and prebuilt artifacts in high-impact open source repositories
- Development of formalised maintainer identity verification standards for critical open source projects
- Empirical study of open source maintainer burnout as a security variable and predictor of supply chain risk
- Extension of the CVE-2024-3094 Research Toolkit to evaluate other supply chain CVEs and generalise the configuration analysis methodology
- Legal and regulatory analysis of how the EU Cyber Resilience Act will affect organisations that consume but do not contribute to open source infrastructure

Appendix A: Affected Distribution and Version Matrix

This appendix provides a comprehensive reference of known affected and unaffected Linux distributions, the protection mechanism where applicable, and remediation status.

Distribution	Affected Versions	Protection Type	Remediation
Debian unstable	5.6.0, 5.6.1	N/A — affected	Rollback to 5.4.6-0.2
Debian testing	5.6.0, 5.6.1	N/A — affected	Rollback to 5.4.6-0.2
Debian experimental	5.6.0, 5.6.1	N/A — affected	Rollback to 5.4.6-0.2
Fedora Rawhide	5.6.0, 5.6.1	N/A — affected	Rollback; system rebuild
Fedora 40 beta	5.6.0	N/A — affected	Rollback; system rebuild
Kali Linux (26–28 Mar 2024)	5.6.0	N/A — affected	Re-image from post-28 Mar image
openSUSE Tumbleweed (specific)	5.6.0, 5.6.1	N/A — affected	Rollback via zypper
Debian Stable	Not affected	Version (5.4.x in stable)	None required
Ubuntu LTS	Not affected	Version (5.4.x in stable)	None required
RHEL / CentOS Stream	Not affected	Version (5.4.x)	None required
Amazon Linux 2/2023	Not affected	Version (5.4.x)	None required
Arch Linux	Not affected	Architecture (no sd_notify patch)	None required
Gentoo	Not affected	Architecture (no sd_notify patch)	None required
NixOS	Not affected	Architecture (no sd_notify patch)	None required
Alpine Linux	Not affected	Architecture (musl libc, no IFUNC)	None required
Void Linux (musl)	Not affected	Architecture (musl libc, no IFUNC)	None required

Appendix B: MITRE ATT&CK Technique Mapping

The following table maps the observed tactics, techniques, and procedures (TTPs) of the XZ Utils supply chain attack to the MITRE ATT&CK Enterprise framework.

ATT&CK ID	Technique	Sub-technique	Observed in Attack
T1078	Valid Accounts	T1078.004 — Cloud Accounts	JiaT75 GitHub account used as legitimate contributor identity
T1585	Establish Accounts	T1585.001 — Social Media Accounts	Sockpuppet accounts (Jigar Kumar, Dennis Ens, others) created on mailing lists
T1195	Supply Chain Compromise	T1195.001 — Compromise Software Dependencies	XZ Utils backdoor delivered via official release tarball
T1554	Compromise Client Software Binary	—	Malicious object file injected into liblzma via build system modification
T1562	Impair Defenses	T1562.001 — Disable or Modify Tools	oss-fuzz IFUNC fuzzing disabled pre-emptively via PR to google/oss-fuzz
T1098	Account Manipulation	—	JiaT75 acquired co-maintainer status and GitHub organisation admin access
T1584	Compromise Infrastructure	T1584.006 — Web Services	XZ project migrated to GitHub organisation under attacker control
T1027	Obfuscated Files or Information	T1027.001 — Binary Padding	Payload concealed in binary test files; RC4 encryption; character substitution obfuscation
T1059	Command and Scripting Interpreter	T1059.004 — Unix Shell	Malicious shell script executed via configure pipeline during build

Appendix C: CVE-2024-3094 Research Toolkit

C.1 Overview

The **CVE-2024-3094 Research Toolkit** is a purpose-built interactive analysis tool developed as an original contribution of this study. It consists of two integrated components: a **System Vulnerability Checker** and an **Attack Surface Visualiser**. The tool is implemented as an HTML single-page application and is provided as a supplementary file accompanying this report.

C.2 System Vulnerability Checker

The System Vulnerability Checker accepts output from three standard Linux diagnostic commands and evaluates the five conditions required for exploitation of CVE-2024-3094. For each condition, it provides a binary PASS/FAIL determination, a technical explanation, and where the system is determined to be vulnerable, remediation commands.

Input Commands

```
# Command 1: XZ Utils version
$ xz --version

# Command 2: sshd dynamic library dependencies
$ ldd /usr/sbin/sshd

# Command 3: SSH daemon active status
$ systemctl is-active ssh
```

Evaluation Logic

The tool evaluates the following conditions in sequence:

Condition	Detection Method	Fail Condition
XZ Utils version	Regex parse of xz --version output	Version matches 5.6.0 or 5.6.1
liblzma linked into sshd	Pattern match for 'liblzma' in ldd output	liblzma.so present in sshd dependency list
libsystemd linked into sshd	Pattern match for 'libsystemd' in ldd output	libsystemd.so present in sshd dependency list
glibc present	Pattern match for 'libc.so', 'glibc', or 'libc-N' in ldd output	glibc detected (IFUNC resolver available)
SSH daemon active	Pattern match for '^active' in systemctl output	sshd reported as active/running

A system is determined **VULNERABLE** if and only if all five conditions evaluate to **FAIL**. If any single condition evaluates to **PASS**, the system is determined **NOT VULNERABLE**, and the specific condition(s) that break the attack chain are identified.

C.3 Attack Surface Visualiser

The Attack Surface Visualiser provides two modes of operation. In its primary mode, it presents an interactive phase-by-phase walkthrough of the attack timeline, with 13 phases from account creation through coordinated response. Each phase updates the dependency chain diagram, displays a contextual code excerpt, and provides ATT&CK framework tags and IAM notes where applicable.

In its secondary mode accessed by clicking 'Visualise this system's dependency chain' after running the System Vulnerability Checker — the visualiser redraws the dependency chain to reflect the actual library profile of the analysed system. Nodes absent from the system's ldd output are dimmed; nodes present are highlighted; the attack chain is annotated to show where it completes or breaks on that specific system.

C.4 Example System Profiles

Profile	XZ Version	liblzma in sshd	libsystemd in sshd	glibc	sshd Active	Verdict
Debian Sid	5.6.1	YES	YES	YES	YES	VULNERABLE
Ubuntu LTS	5.4.6	NO	NO	YES	YES	NOT VULNERABLE
Arch Linux	5.6.0	NO	NO	YES	YES	NOT VULNERABLE

References

Primary Sources

- [1] Freund, A. (2024, March 29). backdoor in upstream xz/liblzma leading to ssh server compromise. oss-security mailing list. <https://www.openwall.com/lists/oss-security/2024/03/29/4>
- [2] National Institute of Standards and Technology. (2024). CVE-2024-3094. National Vulnerability Database. <https://nvd.nist.gov/vuln/detail/CVE-2024-3094>
- [3] Cybersecurity and Infrastructure Security Agency. (2024). Lessons from XZ Utils: Achieving a More Sustainable Open Source Ecosystem. <https://www.cisa.gov/news-events/news/lessons-xz-utils-achieving-more-sustainable-open-source-ecosystem>
- [4] Collin, L. (2024). XZ Utils backdoor. Tukaani Project. <https://tukaani.org/xz-backdoor/>

Academic Sources

- [5] Reith, J., & Zeller, A. (2025). Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain Attack. arXiv:2504.17473. <https://arxiv.org/abs/2504.17473>
- [6] Alvarez, M. et al. (2024). On the critical path to implant backdoors and the effectiveness of potential mitigation techniques: Early learnings from XZ. arXiv:2404.08987. <https://arxiv.org/abs/2404.08987>

Technical Analyses

- [7] Cox, R. (2024). The XZ Utils Backdoor — Timeline. <https://research.swtch.com/xz-timeline>
- [8] Cox, R. (2024). The XZ Utils Backdoor — Shell Script. <https://research.swtch.com/xz-script>
- [9] JFrog Security Research. (2024). XZ Backdoor Attack CVE-2024-3094: All You Need To Know. <https://jfrog.com/blog/xz-backdoor-attack-cve-2024-3094-all-you-need-to-know/>
- [10] Datadog Security Labs. (2024). The XZ backdoor (CVE-2024-3094): Everything you need to know. <https://securitylabs.datadoghq.com/articles/xz-backdoor-cve-2024-3094/>
- [11] Elastic Security Labs. (2024). 500ms to Midnight: XZ Utils Backdoor Analysis. <https://www.elastic.co/security-labs/500ms-to-midnight>
- [12] thesamesam. (2024). XZ Utils Backdoor FAQ. GitHub Gist. <https://gist.github.com/thesamesam/223949d5a074ebc3dce9ee78baad9e27>
- [13] ReversingLabs. (2024). A Software Supply Chain Meltdown: What We Know About the XZ Trojan. <https://www.reversinglabs.com/blog/a-software-supply-chain-meltdown-what-we-know-about-xz-trojan>

Attribution and Social Engineering Analyses

- [14] Kaspersky Global Research and Analysis Team. (2024). XZ backdoor story — Part 1: The technical analysis. Securelist. <https://securelist.com/xz-backdoor-story-part-1/112354/>
- [15] Kaspersky Global Research and Analysis Team. (2024). XZ backdoor story — Part 2: Social engineering. Securelist. <https://securelist.com/xz-backdoor-story-part-2-social-engineering/112476/>
- [16] Kaspersky Global Research and Analysis Team. (2024). XZ backdoor story — Part 3: Hook analysis inside OpenSSH. Securelist. <https://securelist.com/xz-backdoor-part-3-hooking-ssh/113007/>
- [17] Boehs, E. (2024). Everything I Know About the XZ Backdoor. <https://boehs.org/node/everything-i-know-about-the-xz-backdoor>

Vendor Advisories and Distribution Notices

- [18] Red Hat Product Security. (2024). RHSA-2024:1618 — Security Advisory. <https://access.redhat.com/security/cve/CVE-2024-3094>
- [19] Red Hat. (2024). Understanding Red Hat's Response to the XZ Security Incident. <https://www.redhat.com/en/blog/understanding-red-hats-response-xz-security-incident>
- [20] Open Source Security Foundation. (2024). xz Backdoor CVE-2024-3094. <https://openssf.org/blog/2024/03/30/xz-backdoor-cve-2024-3094/>
- [21] Debian Project. (2024). Bug #1067708 — xz-utils: backdoor in versions 5.6.0 and 5.6.1. <https://bugs.debian.org/cgi-bin/bugreport.cgi?bug=1067708>
- [22] Kali Linux. (2024). About the XZ Backdoor. <https://www.kali.org/blog/about-the-xz-backdoor/>
- [23] Arch Linux. (2024). The xz package has been backdoored. <https://archlinux.org/news/the-xz-package-has-been-backdoored/>

Frameworks and Standards

- [24] National Institute of Standards and Technology. (2022). Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations (SP 800-161r1). <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-161r1.pdf>
- [25] MITRE Corporation. (2024). ATT&CK Technique T1195.001 — Supply Chain Compromise: Compromise Software Dependencies and Development Tools. <https://attack.mitre.org/techniques/T1195/001/>
- [26] MITRE Corporation. (2024). ATT&CK Technique T1554 — Compromise Client Software Binary. <https://attack.mitre.org/techniques/T1554/>
- [27] European Parliament and Council of the European Union. (2024). Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act). EUR-Lex. <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R2847>